

MONITOR-X

Automated API Testing, Monitoring & Reliability Platform

Final Production Specification, System Design & Deployment Guide

Version 1.0 - Deployment-ready architecture

Prepared for the Monitor-X project team

Nandik Dawar | Harsh Kumar Jha | Shruti Tripathi

University of Petroleum & Energy Studies, Dehradun

August 2026

Purpose of this document

This report converts the original 43-page project synopsis into an implementation specification that can be handed to a development team and used to build, secure, test, deploy and operate the final Monitor-X system. Source-derived requirements are preserved; production additions are explicitly marked as final architecture decisions.

Document Control

Field	Value
Document	Monitor-X Final Production Specification, System Design & Deployment Guide
Version	1.0
Status	Approved design baseline for implementation
Baseline source	MonitorX_report.pdf, Project Synopsis dated 18/08/2026
Primary deployment target	AWS, with a lower-cost portfolio deployment profile also defined
Primary audience	Developers, reviewers, DevOps engineers, QA engineers and viva/interview evaluators

How to read this report

Sections 1-4 define what the product must do. Sections 5-12 explain how it will be built. Sections 13-18 define deployment, testing and operations. Sections 19-22 provide the implementation roadmap, risk register and completion criteria. The appendices contain concrete configuration templates and examples.

Source baseline versus final additions

The original synopsis already defines API test configuration, manual and scheduled execution, response validation, monitoring, historical storage, failure logging, dashboards and alerts. It also identifies React, Node.js/Express, a custom test execution engine, a scheduling system, a database and a Redis/job-queue layer. This final report keeps that intent while replacing or strengthening selected implementation choices required for a production-grade build.

Area	Baseline synopsis	Final production decision
Frontend	React web interface	React + TypeScript + Vite + Tailwind + TanStack Query + Recharts
Backend	Node.js / Express	Node.js + Express + TypeScript; stateless API service
Database	MongoDB	PostgreSQL + Prisma; JSONB only where flexible request/test data benefits from it
Scheduling	Scheduler + Redis/job queue	Redis + BullMQ delayed jobs, reconciliation process, deterministic job IDs and worker pools
Monitoring	Dashboard + historical metrics	Time-series aggregation from PostgreSQL plus live events over WebSockets
Alerts	Alerting mechanism	Slack, email and signed webhook channels with dedupe, retries and cooldowns
Deployment	Cloud/VPS dependent	Containerized AWS architecture with ECS Fargate, RDS, ElastiCache, S3/CloudFront, ECR, Secrets Manager, CloudWatch and WAF
Security	Authentication, authorization and credential protection	Multi-tenant RBAC, encrypted secrets, SSRF protection, scoped API keys, audit logs and strict outbound-request controls

Contents

1. Product Vision, Goals and Scope
 2. Personas and User Journeys
 3. Functional Requirements
 4. Non-functional Requirements and SLOs
 5. Final System Architecture
 6. API Test Execution Engine
 7. Scheduling, Queues and Distributed Workers
 8. Data Architecture and PostgreSQL Schema
 9. Backend API Contract
 10. Frontend and UX Specification
 11. Security Architecture
 12. Observability, Analytics and Incident Management
 13. Testing and Quality Strategy
 14. Local Development and Repository Structure
 15. AWS Deployment Architecture
 16. CI/CD and Release Management
 17. Backup, Disaster Recovery and Operations
 18. Environment Strategy and Cost Control
 19. Implementation Roadmap
 20. Definition of Done and Acceptance Criteria
 21. Risk Register
 22. CV / Portfolio Positioning
- Appendices A-F: configuration, Docker, CI/CD and sample payloads

1. Product Vision, Goals and Scope

1.1 Product vision

Monitor-X is a developer-focused web platform for defining API checks, executing them on demand or on a schedule, validating responses, measuring availability and latency, preserving historical results, detecting incidents and notifying teams. The final product combines the useful parts of Postman-style request testing, uptime monitoring and lightweight incident management in one system.

1.2 Primary goals

- Reduce repetitive API verification by turning reusable request definitions into automated checks.
- Detect outages, regressions and performance degradation after deployment instead of only during development.
- Provide a trustworthy historical record of status, latency, assertions and incident state.
- Support teams and multiple environments without exposing API credentials.
- Run scheduled checks asynchronously so dashboard/API traffic is never blocked by monitored requests.
- Be demonstrably deployable: containerized services, reproducible infrastructure, secure secrets, health checks, telemetry, backups and a documented rollback path.

1.3 Supported API styles

Style	Support approach	Final scope
REST / JSON	Native request builder and JSON assertion engine	Must-have
GraphQL	HTTP POST/GET with query + variables; JSON response assertions	Must-have
SOAP / XML	Generic HTTP request with XML body; XPath/text assertions	Supported without a separate SOAP runtime
Arbitrary HTTP	Custom method, headers, parameters and body	Supported where safe

1.4 Final scope

- Authentication and team workspaces
- Projects, collections and environments
- Reusable API test builder
- Manual runs and collection runs
- Scheduled monitoring
- Response assertions and latency thresholds
- Historical metrics and uptime
- Incidents and alerts
- OpenAPI/Swagger import
- Public API keys and CI/CD trigger support
- Audit logs and RBAC
- AWS deployment, logging, metrics, backups and release automation

1.5 Explicitly out of scope for version 1.0

- Browser/UI testing and Selenium orchestration
- Full synthetic browser journeys
- High-volume load testing of customer APIs
- APM agents inside customer applications
- Infrastructure monitoring for CPU, disk and hosts
- Distributed global probes from many countries
- Automatic mutation/fuzz testing of APIs

Scope rule

A final feature is accepted only if it strengthens API testing, reliability monitoring, developer workflow, security, deployability or observability. Features that turn Monitor-X into a generic project-management product are rejected.

2. Personas and User Journeys

Persona	Needs	Key actions
Developer	Fast request testing and regression visibility	Create test, run now, inspect body/assertions, compare history
Backend/API engineer	Availability and performance signals	Schedule checks, set latency limits, inspect failures and incidents
QA engineer	Repeatable API validation	Create collections, configure expected results, run suites, export/review results
Team admin	Secure collaboration	Invite users, set roles, configure environments, manage alert channels and API keys
Viewer / stakeholder	Simple health overview	View dashboards, uptime, incidents and recent failures

2.1 Core user journey

1. Create an organization/workspace and a project.
2. Create an environment such as Development, Staging or Production.
3. Add non-secret variables and encrypted secrets.
4. Create a collection and define one or more API tests, or import an OpenAPI specification.
5. Configure assertions and an optional latency threshold.
6. Run the test manually and inspect the execution detail.
7. Create a recurring schedule.
8. Monitor uptime, response latency, pass rate and failures.
9. Receive an alert when incident rules are met.
10. Acknowledge/investigate/resolve the incident and review history.

3. Functional Requirements

3.1 Authentication and account security

- Email/password authentication with verified email, secure password hashing and account lockout/backoff.
- Optional GitHub OAuth for developer convenience.
- Short-lived access session and rotating refresh session stored in Secure, HttpOnly, SameSite cookies.
- Password reset and email verification tokens are single-use, time-limited and hashed at rest.
- All authentication events are audit logged.

3.2 Organizations, roles and tenancy

Role	Permissions
Owner	Full organization control, billing/configuration, delete org, manage owners/admins
Admin	Manage projects, members, environments, alert channels and API keys
Editor	Create/edit tests, collections, schedules and run tests
Viewer	Read dashboards, tests, history and incidents only

Every organization-owned object is tenant-scoped. Authorization must verify tenant membership on every request. Object IDs alone must never grant access.

3.3 Projects, collections and tests

- Project groups one monitored product/service.
- Collection groups logically related API tests and can be executed as a suite.
- A test stores method, URL template, query parameters, headers, body template, authentication configuration, timeout, redirect policy, assertions and optional tags.
- Tests are versioned logically using updated_at and audit events; high-value changes such as endpoint, auth or assertions are captured in a change history.

3.4 Request builder

Capability	Requirement
Methods	GET, POST, PUT, PATCH, DELETE, HEAD, OPTIONS; custom method only for advanced mode
URL	Template variables allowed; final expanded URL validated before execution
Query	Key/value rows with enabled/disabled flag and variables
Headers	Key/value rows; sensitive values can reference secrets
Body	none, JSON, raw text/XML, x-www-form-urlencoded; multipart can be added later
Authentication	Bearer token, Basic auth, API key header/query, custom header and no-auth
Timeout	Per-test, bounded by organization maximum
Redirects	Disabled by default for monitors; configurable up to a small maximum with validation on every hop

3.5 Environments and secrets

- Variables are referenced using a deterministic syntax such as {{BASE_URL}}.
- Secret values are encrypted at rest and masked in the UI after creation.
- Secrets are never returned in API responses after they are saved.
- Authorization, Cookie, Set-Cookie and user-configured sensitive headers are redacted from logs and stored execution previews.
- Environment selection is explicit for manual and scheduled runs.

3.6 Assertion engine

Assertion type	Example
Status	status == 200; status in [200,201,204]
Latency	total_ms < 500
Header	content-type contains application/json
JSON path exists	\$.user.id exists
JSON comparison	\$.success == true; \$.items.length > 0
Text	body contains "healthy"
Regex	body matches a configured regular expression with execution limits
XML/XPath	/Envelope/Body/Status == OK
Schema - optional advanced	Validate JSON against supplied JSON Schema

Each assertion produces an independent assertion result so the execution detail can show exactly what passed and failed. One failed required assertion marks the test failed. Soft/warning assertions may mark the test degraded without opening an incident unless configured.

3.7 Manual execution

- Run single test immediately.
- Run a collection sequentially or with bounded parallelism.
- Cancel queued/running jobs where technically safe.
- Return a run ID immediately; browser receives live updates over WebSocket/SSE.
- Execution history is filterable by time, status, environment and tag.

3.8 Scheduled monitoring

- Intervals: 1, 5, 10, 15, 30 and 60 minutes plus hourly/daily schedules. Very aggressive intervals can be plan/admin controlled.
- Schedules are stored in PostgreSQL as the source of truth.
- BullMQ delayed jobs trigger execution; no user HTTP request waits for the monitored API.
- Missed or duplicate jobs are handled through reconciliation and idempotency keys.
- Paused schedules do not count toward uptime.

3.9 Health model

State	Meaning
HEALTHY	Latest eligible checks pass and latency is within configured limits.
DEGRADED	API responds but warning assertion/latency threshold is violated.
DOWN	Required assertions fail or request cannot complete under the configured policy.
PAUSED	Monitoring intentionally disabled or inside maintenance window.
UNKNOWN	No recent eligible execution or system cannot determine current state.

3.10 Incidents

- Configurable incident rule, e.g. open after 2 consecutive failures or immediately for a critical assertion.
- Deduplicate repeated failures into the existing open incident.
- Incident states: OPEN, ACKNOWLEDGED, INVESTIGATING, RESOLVED.
- Auto-resolve after a configured number of consecutive successful checks.
- Timeline records state changes, alerts sent, execution IDs and human notes.

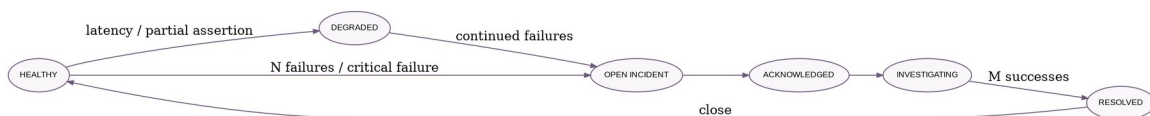


Figure 1. Monitor-X incident and health lifecycle

3.11 Alerting

- Channels: email, Slack incoming webhook/app integration and generic signed webhook.
- Trigger on incident opened, optionally degraded state, recovery and repeated unresolved incident reminders.
- Per-channel cooldown avoids message storms.
- Delivery failures retry with exponential backoff and become visible in the dashboard.
- Generic webhooks include timestamp, event ID and HMAC signature so receivers can verify authenticity.

3.12 OpenAPI / Swagger import

- Accept OpenAPI 3.x JSON or YAML file/URL.
- Parse paths, methods, servers, parameters and example request bodies.

- Generate one draft Monitor-X test per operation; never auto-enable production schedules.
- Map servers to environment variables when possible.
- Mark unresolved authentication schemes or required examples for user review.

3.13 CI/CD and public API integration

A scoped Monitor-X API key allows automation to trigger a test or collection and poll the run result. A small CLI can wrap this API for GitHub Actions. CI mode returns a non-zero process exit when required assertions fail.

```
monitorx run collection payment-service --environment staging --wait
# Exit 0: all required checks passed
# Exit 1: one or more checks failed
# Exit 2: configuration/authentication error
```

3.14 Audit and administration

- Audit: member invites/role changes, API key creation/revocation, test/schedule changes, environment secret changes, incident state changes.
- API keys are displayed only once at creation and stored as cryptographic hashes.
- Admin can disable a schedule, revoke a key and remove a member without database access.

4. Non-functional Requirements and SLOs

4.1 Internal service-level objectives

SLO	Target for production profile
Control-plane API availability	99.9% monthly, excluding planned maintenance
API read latency	p95 < 400 ms for ordinary dashboard/API queries under expected load
Schedule dispatch delay	95% of due checks start within 60 seconds of scheduled time
Execution persistence	A completed worker run is persisted before it is considered successful
Alert dispatch	95% of incident-open alerts are enqueued within 30 seconds of state change
Recovery	An API/worker restart does not lose schedule definitions or historical execution records

Important

These SLOs apply to Monitor-X itself. The target APIs being monitored are outside Monitor-X control. Target response times and availability are measured, not guaranteed.

4.2 Scalability goals

- API service remains stateless so it can scale horizontally behind a load balancer.
- Workers scale independently from the API service based on queue depth and job age.
- All expensive outbound HTTP work occurs in workers, not web request handlers.
- Per-organization and per-target-host concurrency limits protect Monitor-X and external APIs.
- Historical execution queries use indexed, paginated access and pre-aggregated metrics where needed.

4.3 Reliability principles

- PostgreSQL is the source of truth for configuration and durable business state.
- Redis accelerates queues/cache but loss of Redis must not permanently lose schedules; reconciliation can rebuild future jobs.
- Every scheduled execution has a deterministic idempotency key.
- Retries are limited and explicit; retries must not silently create duplicate incidents or metrics.
- Deployments use readiness checks and rolling replacement.

4.4 Data retention defaults

Data	Default retention	Notes
Execution metadata	90 days	Configurable by deployment/plan
Response preview	7 days	Truncated and redacted; may be disabled
Full response artifact	Off by default	Optional encrypted S3 object with short TTL
Incidents	1 year	Useful for reliability trend analysis
Audit logs	1 year	Security/administrative trace
Queue job metadata	Short operational retention	Not a substitute for execution records

5. Final System Architecture

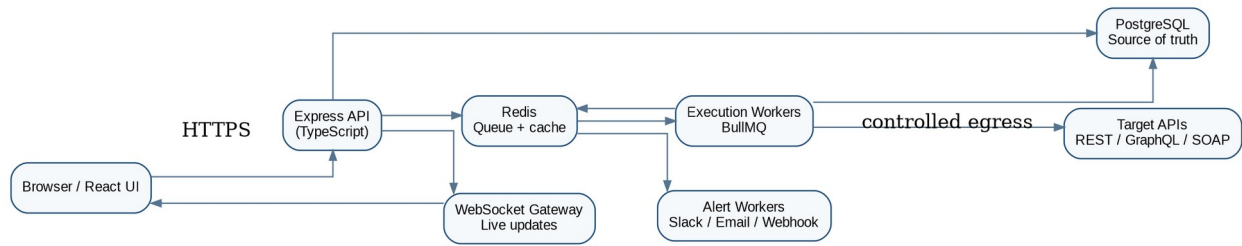


Figure 2. Final logical architecture

5.1 Component responsibilities

Component	Responsibility
React web app	Dashboard, test builder, results, incidents, team settings and live updates
Express API	Authentication, authorization, CRUD, aggregation, run submission and WebSocket authentication
PostgreSQL	Durable source of truth for organizations, tests, schedules, executions, incidents, alerts and audit records
Redis + BullMQ	Delayed/scheduled work, worker queues, short-lived locks, rate limiting and ephemeral state
Execution workers	Safely execute outbound requests, assertions and metrics collection
Alert workers	Deliver Slack/email/webhook notifications asynchronously
WebSocket gateway	Push run status, incident state and dashboard changes to connected clients
Object storage	Optional OpenAPI files and short-lived large response artifacts

5.2 Architecture decisions

ADR	Decision	Reason
ADR-001	Use PostgreSQL instead of MongoDB for the final build	The domain is highly relational: organizations, memberships, projects, tests, schedules, executions, assertions, incidents and alert deliveries. JSONB still handles flexible request/assertion configuration.
ADR-002	Use BullMQ on Redis for background work	Delayed jobs, retries, backoff, concurrency and worker separation are core requirements.
ADR-003	Keep API stateless; workers perform outbound tests	Improves responsiveness and enables independent scaling/security boundaries.
ADR-004	Use AWS ECS Fargate for production containers	Separates API and workers without managing servers and supports rolling deployment.
ADR-005	Use AWS Secrets Manager/KMS for infrastructure secrets and application-level encryption for tenant secrets	Avoids plaintext credentials and supports controlled key rotation.
ADR-006	Use structured logs + OpenTelemetry/CloudWatch	Makes the monitoring platform itself observable and diagnosable.

6. API Test Execution Engine

6.1 Execution contract

A worker receives a job containing only identifiers and immutable execution metadata. It reloads the current authorized test/environment configuration from PostgreSQL, expands variables, validates the final URL, applies outbound restrictions and executes the request. Decrypted secrets exist only in worker memory for the duration of the run.

6.2 Request safety limits

Limit	Recommended default
Connection + total timeout	10 seconds total; configurable up to 30 seconds
Maximum response body read	1 MB hard cap; store only configured preview, e.g. 64 KB
Redirects	0 by default; max 3-5 if enabled, validating every redirect target
Request body	1 MB default cap
Header count/size	Bounded to prevent abuse
TLS	Certificate validation on; no insecure bypass in production

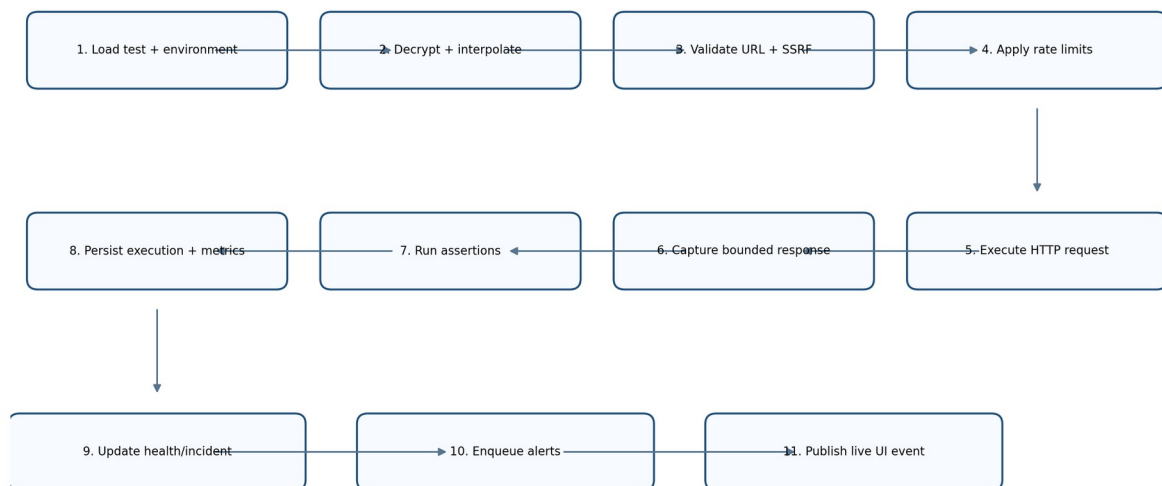


Figure 3. Outbound test execution lifecycle

6.3 SSRF protection - mandatory

Critical security requirement

Monitor-X accepts user-controlled URLs and performs server-side outbound requests. Without SSRF defenses an attacker could probe private networks, cloud metadata services, databases or internal admin endpoints. This is the highest-risk feature in the product.

- Allow only http/https schemes.
- Resolve hostnames before the request and reject loopback, link-local, private, multicast, carrier-grade NAT and reserved ranges for both IPv4 and IPv6.
- Reject cloud metadata/link-local targets such as 169.254.0.0/16.
- Re-validate every redirect destination.
- Protect against DNS rebinding by using a controlled DNS lookup/connection strategy and validating the resolved address used by the socket.
- Apply organization-level allowlists when monitoring private/internal APIs is intentionally supported in a dedicated private-agent architecture; do not weaken the public SaaS worker policy.

- Restrict outbound worker security groups/NACLs where practical and keep workers isolated from sensitive control-plane endpoints.

6.4 Assertion processing

Assertions run after the response is bounded and decoded. The engine normalizes errors into `configuration_error`, `network_error`, `timeout`, `tls_error`, `response_error` and `assertion_failure` classes. Assertions must never execute arbitrary user JavaScript in the worker process.

6.5 Execution result model

```
{
  "executionId": "ex_123",
  "testId": "test_42",
  "scheduledAt": "2026-08-30T12:00:00Z",
  "startedAt": "2026-08-30T12:00:02Z",
  "finishedAt": "2026-08-30T12:00:02.284Z",
  "status": "FAILED",
  "httpStatus": 503,
  "latencyMs": 284,
  "responseBytes": 431,
  "assertions": [
    {"type": "status", "passed": false, "expected": 200, "actual": 503},
    {"type": "latency", "passed": true, "expected": "<500", "actual": 284}
  ],
  "errorClass": null
}
```

6.6 Retry policy

- Automatic job retry is for Monitor-X operational failures, e.g. transient worker/Redis/database connectivity, not for a legitimate target API 500 response.
- A target 500/timeout is normally recorded as the actual monitoring result.
- Operational retries use capped exponential backoff with jitter.
- Retried attempts are associated with one logical scheduled execution to avoid double-counting uptime.

7. Scheduling, Queues and Distributed Workers

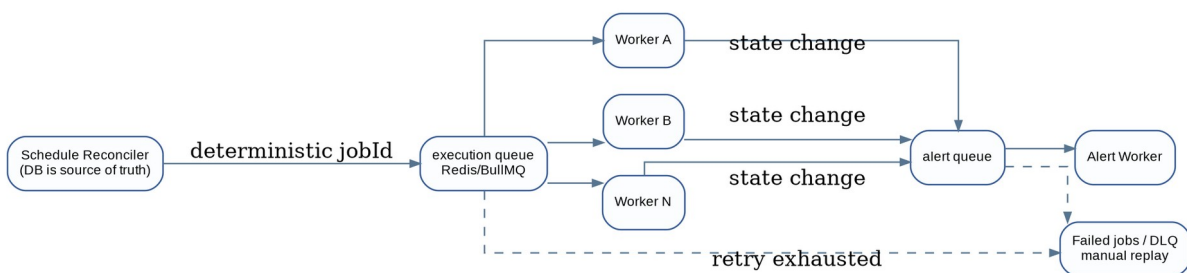


Figure 4. Scheduling and worker topology

7.1 Queue set

Queue	Purpose	Typical workers
execution	Manual and scheduled API checks	HTTP execution workers
collection-run	Fan-out collection runs with bounded parallelism	Coordinator worker
alerts	Slack/email/webhook delivery	Alert workers
imports	OpenAPI parsing and test draft generation	Import worker

Queue	Purpose	Typical workers
maintenance	Retention cleanup, aggregation and reconciliation	Maintenance worker

7.2 Scheduling model

Schedule rows live in PostgreSQL with cadence, timezone, enabled flag and next_run_at. The scheduler/reconciler creates BullMQ delayed jobs using a deterministic key such as schedule:{scheduleId}:{scheduledAt}. After a scheduled job is accepted, next_run_at is advanced transactionally or during a reconciled state transition.

7.3 Reconciliation

- On scheduler startup and periodically, query enabled schedules whose next_run_at is due or near-future.
- Ensure the corresponding deterministic BullMQ job exists.
- If Redis was cleared, rebuild jobs from PostgreSQL.
- If the job exists, do nothing; duplicate enqueue attempts are harmless.
- Track scheduler lag and number of reconciled/missed schedules as internal metrics.

7.4 Concurrency and rate limiting

- Global worker concurrency protects infrastructure.
- Per-organization concurrency prevents one tenant consuming all workers.
- Per-target-host concurrency avoids hammering the same external API.
- Optional requests-per-minute/hour limits use Redis token bucket/sliding window counters.
- When a tenant limit is reached, reschedule the job rather than busy-waiting.

7.5 Idempotency

The unique tuple (test_id, scheduled_at, run_kind) identifies a logical scheduled execution. Database uniqueness plus deterministic queue job IDs prevents duplicate history rows and duplicate incidents after retries or worker restarts.

8. Data Architecture and PostgreSQL Schema

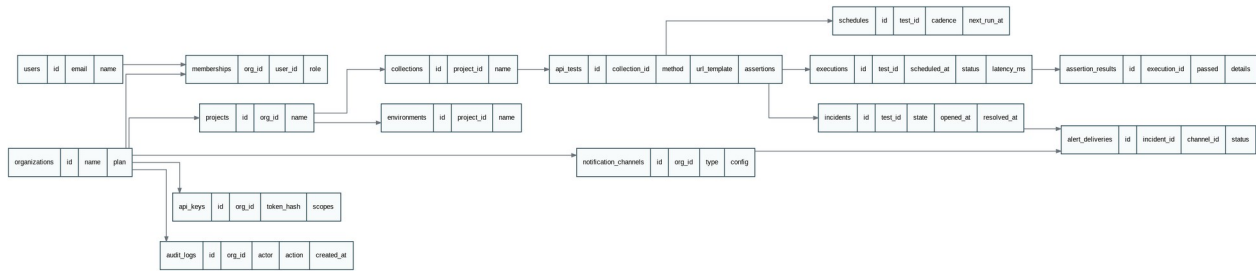


Figure 5. Core relational data model

8.1 Core entities

Table	Important columns / purpose
users	id, email, name, password_hash/oauth identity, status, created_at
organizations	id, name, plan/status, created_at
memberships	organization_id, user_id, role; unique membership
projects	id, organization_id, name, slug
collections	id, project_id, name, description
environments	id, project_id, name, variables_json; secret values stored separately/encrypted
environment_secrets	environment_id, key, ciphertext, key_version
api_tests	id, collection_id, name, method, url_template, request_config_json, assertion_config_json, timeout_ms
schedules	id, test_id, environment_id, cadence, timezone, enabled, next_run_at
executions	id, test_id, environment_id, run_kind, scheduled_at, started_at, finished_at, status, http_status, latency_ms, error_class, response_preview
assertion_results	execution_id, assertion_index/type, passed, expected_json, actual_json, message
incidents	id, test_id, environment_id, state, severity, opened_at, acknowledged_at, resolved_at, latest_execution_id
incident_events	incident_id, event_type, actor, execution_id, payload_json, created_at
notification_channels	organization_id, type, name, encrypted_config
alert_deliveries	incident_id, channel_id, event_type, status, attempts, last_error, delivered_at
api_keys	organization_id, name, token_hash, token_prefix, scopes, expires_at, revoked_at
audit_logs	organization_id, actor_user_id, action, object_type, object_id, metadata_json, created_at

8.2 Indexing

- executions(test_id, started_at desc)
- executions(environment_id, started_at desc)
- executions(status, started_at desc) for failure filters
- schedules(enabled, next_run_at)
- incidents(test_id, environment_id, state)
- audit_logs(organization_id, created_at desc)
- alert_deliveries(status, created_at) for retry/ops views

8.3 Partitioning and aggregation

For portfolio and moderate usage, a normal indexed executions table is sufficient. At higher scale, partition executions by month and maintain hourly/daily aggregate tables for dashboard ranges. Do not prematurely introduce a separate time-series database unless measured load justifies it.

8.4 Secret storage

Infrastructure secrets (database password, OAuth client secret, SMTP/API credentials) live in AWS Secrets Manager. Tenant environment secrets are encrypted with an application-managed data-encryption key protected by KMS. Store ciphertext, nonce/IV, algorithm and key version; never plaintext. Rotation must support decrypting old values while newly written values use the newest key.

9. Backend API Contract

9.1 API conventions

- Base path /api/v1.
- JSON request/response unless importing a file.
- Stable error envelope with machine-readable code and request ID.
- Cursor or page-based pagination on list endpoints; never return unbounded execution history.
- Every mutation checks tenant role and writes an audit event when security/behavior changes.

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "The test URL is invalid.",
    "requestId": "req_abc123",
    "details": [{"field": "urlTemplate", "reason": "required"}]
  }
}
```

9.2 Endpoint map

Method	Path	Purpose
POST	/auth/register	Create account
POST	/auth/login	Login
POST	/auth/refresh	Rotate session
POST	/auth/logout	Revoke current refresh session
GET	/organizations	List memberships
POST	/organizations	Create organization
POST	/organizations/:id/invites	Invite member
PATCH	/organizations/:id/members/:userId	Change role
GET	/projects	List projects in current organization
POST	/projects	Create project
GET	/collections/:id	Get collection with tests
POST	/collections	Create collection
POST	/tests	Create API test
PATCH	/tests/:id	Update test
POST	/tests/:id/run	Queue manual run
GET	/tests/:id/executions	Paginated history
GET	/executions/:id	Execution + assertion detail
POST	/collections/:id/run	Queue collection run
POST	/schedules	Create schedule
PATCH	/schedules/:id	Pause/update schedule
GET	/metrics/tests/:id	Aggregated uptime/latency series
GET	/incidents	List incidents
PATCH	/incidents/:id/state	Acknowledge/investigate/resolve
POST	/notification-channels	Create alert channel
POST	/notification-channels/:id/test	Send test notification
POST	/imports/openapi	Upload/import specification
POST	/api-keys	Create scoped key
DELETE	/api-keys/:id	Revoke key
GET	/audit-logs	Organization audit history

9.3 Automation endpoint

POST /api/v1/automation/collections/{collectionId}/runs

Authorization: Bearer mx_live_XXXXX

```
{
  "environmentId": "env_staging",
  "wait": false,
  "source": "github-actions",
  "commitSha": "abc123"
}
```

9.4 WebSocket events

Event	Payload
execution.queued	executionId, testId, runKind
execution.started	executionId, startedAt
execution.completed	executionId, status, latencyMs, httpStatus
incident.opened	incidentId, testId, severity
incident.updated	incidentId, state
alert.delivery	incidentId, channelId, status

10. Frontend and UX Specification

10.1 Main routes

Route	Purpose
/login, /register	Authentication
/app	Organization overview dashboard
/app/projects/:id	Project health summary
/app/collections/:id	Collection tests and run controls
/app/tests/:id	Test builder + health summary
/app/executions/:id	Request/response/assertion detail
/app/incidents	Incident queue and timeline
/app/alerts	Notification channels and delivery history
/app/team	Members, roles and invites
/app/settings/environments	Variables and secrets
/app/settings/api-keys	Automation keys
/app/audit	Audit log

10.2 Test builder tabs

- Request: method, URL, query parameters, headers and auth.
- Body: JSON/raw/XML/form encoding.
- Assertions: status, latency, headers, JSON/XML/text checks.
- Environment: select environment and preview variable substitution with secrets masked.
- Schedule: create/pause monitor and incident threshold.
- History: previous executions and quick comparison.

10.3 Dashboard metrics

- Total monitored tests and current healthy/degraded/down counts.
- Uptime percentage over 24h/7d/30d.
- Average and p50/p95/p99 latency.
- Recent incidents and currently open incidents.
- Failure rate and error class breakdown.
- Most degraded endpoints and slowest endpoints.

10.4 Accessibility and usability

- Keyboard-accessible forms and dialogs.
- Status never communicated by color alone; always pair icon/text.
- Tables support pagination, filtering and responsive overflow.
- Code/JSON viewers use monospace font, copy button and sensitive-value masking.
- Destructive actions require explicit confirmation.

11. Security Architecture

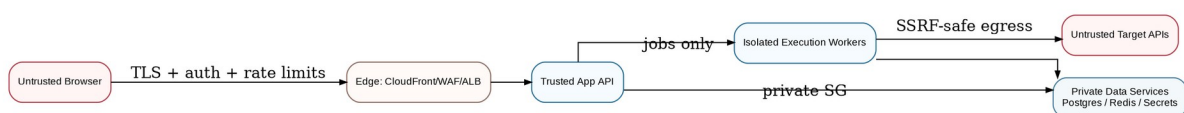


Figure 6. Trust boundaries and network security model

11.1 Application security controls

Threat	Control
Broken tenant isolation	Organization-scoped service methods, membership checks, tests for cross-tenant access

Threat	Control
Credential exposure	KMS-backed encryption, redaction, no secret echo, minimal response storage
SSRF / cloud metadata access	URL policy, DNS/IP validation, redirect revalidation, egress isolation
XSS	React escaping, Content Security Policy, no unsanitized HTML rendering
CSRF	SameSite cookies plus CSRF token/origin checks for state-changing cookie-auth requests
SQL injection	Parameterized ORM queries; no concatenated raw SQL
Brute-force auth	Rate limiting, secure hashing, lockout/backoff, MFA-ready design
API abuse	Per-IP/user/org rate limits and scoped API keys
Webhook spoofing	HMAC signatures + timestamp + replay window
Sensitive logs	Structured logger redaction for auth headers, cookies, tokens and configured secret keys

11.2 API key format and storage

Generate at least 32 bytes of cryptographically random material. Show the full token only once. Store a short prefix for identification and a strong hash of the token for verification. Keys carry scopes and optional expiration. Revocation is immediate.

11.3 Security headers

- Strict-Transport-Security after HTTPS rollout is stable.
- Content-Security-Policy for the web application.
- X-Content-Type-Options: nosniff.
- Referrer-Policy and Permissions-Policy with restrictive defaults.
- No wildcard CORS for authenticated endpoints.

11.4 Data minimization

Monitor-X should record the minimum data required to diagnose an API check. Do not persist full Authorization/Cookie headers. Response storage is truncated and redacted. Users can disable response body retention for sensitive production endpoints.

12. Observability, Analytics and Incident Management

12.1 Monitor-X internal observability

- Structured JSON logs with requestId, organizationId where safe, service, jobId and executionId.
- OpenTelemetry traces for inbound API calls, queue submission, worker execution and database operations.
- CloudWatch metrics/alarms for API 5xx rate, p95 latency, worker failures, queue depth, oldest job age, scheduler lag, database connections and alert delivery failure.
- Separate internal operational dashboards from customer target-API dashboards.

12.2 Customer-facing reliability metrics

Metric	Calculation
Availability %	successful eligible monitor checks / total eligible monitor checks x 100
Pass rate	executions passing all required assertions / total executions
Latency p50/p95/p99	Percentiles over eligible execution latency in selected time range
MTTR	Average time from incident open to resolved
Incident count	Number of incidents opened in selected period
Error breakdown	Network, timeout, TLS, HTTP status and assertion failure counts

12.3 Incident rules

- Default: open after 2 consecutive DOWN checks; critical tests can open after 1.
- Resolve after 2 consecutive healthy checks.
- A DEGRADED status can alert without opening an incident unless user enables it.
- Maintenance windows suppress incident opening/alerts but can optionally keep raw execution history.

13. Testing and Quality Strategy

13.1 Test pyramid

Layer	Tools / focus
Unit	Vitest/Jest: validators, assertion engine, schedule calculations, auth helpers, rate limits
Integration	Supertest + test PostgreSQL/Redis: API routes, repositories, queue behavior, tenant isolation
Worker integration	Mock HTTP targets: timeouts, redirects, large body, DNS/IP restrictions, retries, assertion persistence
End-to-end	Playwright: login, create test, run, schedule, view result, incident workflow
Load/performance	k6: API read endpoints, run submission, WebSocket fan-out under expected usage
Security	Dependency audit, SAST, secret scanning, OWASP ZAP against staging; dedicated SSRF regression tests

13.2 Mandatory regression cases

- User from organization A cannot read/update any object in organization B.
- Target URL resolves to 127.0.0.1/private/link-local/metadata address and is rejected.
- Redirect from public URL to private IP is rejected.
- Authorization/Cookie headers never appear in stored logs or UI previews.
- Redis is flushed/restarted and future schedules are rebuilt from PostgreSQL.
- Worker crashes after outbound call but before ack; idempotency prevents duplicate logical execution.
- Alert provider returns 500; delivery retries without duplicating the incident.
- Database migration is backward-compatible with rolling deployment or deployment is sequenced to avoid mixed-version failure.

13.3 Pre-release quality gate

- Lint and typecheck pass.
- Unit/integration test suite passes.
- Docker images build from clean checkout.
- Database migration applies to a copy of staging data.
- Staging smoke test executes a known healthy and known failing monitor.
- SSRF regression suite passes.
- No high-severity dependency vulnerabilities without documented exception.

14. Local Development and Repository Structure

14.1 Recommended monorepo

```
monitor-x/
apps/
  web/      # React + Vite + TypeScript
  api/      # Express API + WebSocket gateway
  worker/   # BullMQ execution/alert/import workers
packages/
  db/       # Prisma schema, migrations, repositories
  contracts/ # Zod schemas and shared API types
  test-engine/ # request normalization + assertion engine
  security/  # SSRF, encryption, redaction helpers
  ui/       # optional shared components
  cli/      # monitorx CLI for CI/CD
infra/
  cdk/      # AWS CDK TypeScript stacks
docker/
.github/workflows/
docs/
```

14.2 Local services

- PostgreSQL container
- Redis container
- Mail testing service such as Mailpit for development
- API, worker and web dev servers
- Optional local mock target API that exposes healthy, slow, error, redirect and large-response endpoints

14.3 Developer commands

```
pnpm install
pnpm db:migrate
pnpm dev      # web + api + worker
pnpm test
pnpm test:integration
pnpm test:e2e
pnpm lint
pnpm typecheck
pnpm build
```

15. AWS Deployment Architecture

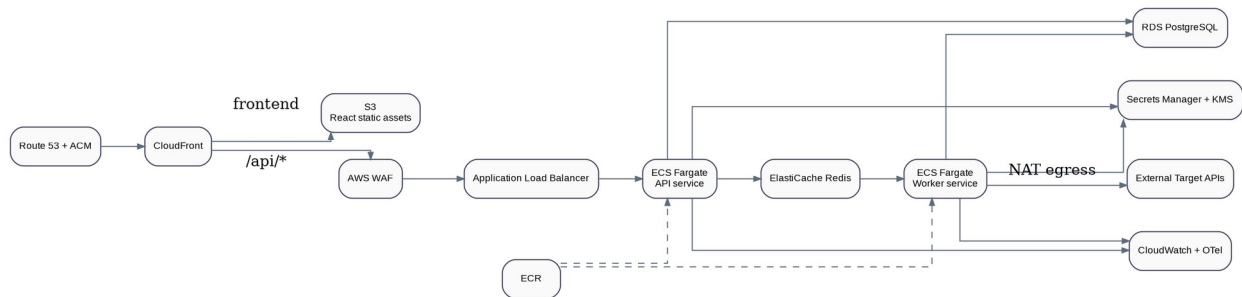


Figure 7. Production AWS deployment target

15.1 Production topology

AWS service	Use
Route 53	DNS for app and API domains
ACM	TLS certificates
S3 + CloudFront	Static React build and edge caching
AWS WAF	Basic web/API protection and managed rules
ALB	Public HTTPS ingress to the API ECS service
ECS Fargate	API service, execution workers, alert/import/maintenance workers
ECR	Immutable container images
RDS PostgreSQL	Durable relational database; production uses automated backups and Multi-AZ when budget requires
ElastiCache Redis	BullMQ queues, locks and rate limiting
Secrets Manager + KMS	Infrastructure secrets and encryption-key protection
CloudWatch	Logs, metrics, dashboards and alarms
SES or provider API	Email notifications

15.2 Network design

- VPC across at least two Availability Zones for production.
- ALB in public subnets.
- API, workers, RDS and Redis in private subnets.
- Only ALB security group can reach API service port.
- Only API/worker security groups can reach RDS/Redis.
- Execution workers require controlled internet egress through NAT for target APIs.
- RDS and Redis have no public endpoint.

15.3 ECS services

Service	Ingress	Autoscaling signal
api	ALB HTTPS only	CPU/memory plus request count/latency
worker-execution	No public ingress	Queue depth + oldest execution-job age
worker-alerts	No public ingress	Alert queue depth
worker-maintenance	No public ingress	Usually one task; schedule/reconciliation lock ensures singleton behavior

15.4 Infrastructure as Code

Use AWS CDK in TypeScript so infrastructure is versioned with the application. Suggested stacks: NetworkStack, DataStack, EdgeStack, ComputeStack and ObservabilityStack. Production resources must be reproducible from source control; manual console changes are exceptions, not the normal deployment path.

15.5 Deployment order

1. Create/verify Route 53 hosted zone and TLS certificates.
2. Deploy VPC/network and security groups.
3. Deploy RDS and Redis; store credentials in Secrets Manager.
4. Create ECR repositories and build/push images.
5. Run database migration as a one-off controlled task.
6. Deploy ECS API and workers with health checks.
7. Deploy S3/CloudFront frontend.
8. Configure WAF, DNS and API origin.
9. Create CloudWatch alarms and notification destinations.

10. Run smoke tests and a controlled failing monitor.
11. Verify backup configuration and rollback procedure.

16. CI/CD and Release Management



Figure 8. Continuous integration and deployment pipeline

16.1 Pull request checks

- Install from lockfile.
- Lint and TypeScript typecheck.
- Unit and integration tests with ephemeral PostgreSQL/Redis services.
- Build web/API/worker.
- Run dependency and secret scan.

16.2 Main branch deployment

1. Tag build with commit SHA.
2. Build API/worker images once and push immutable tags to ECR.
3. Run database migration task.
4. Update ECS task definitions to the new image tag.
5. Wait for rolling deployment and ALB readiness.
6. Upload web assets to S3 and invalidate required CloudFront paths.
7. Run staging/production smoke checks.
8. If health checks fail, redeploy previous known-good task definition/image tag.

16.3 Migration rule

Release safety

Schema changes must be expand-and-contract where possible: add compatible fields/tables first, deploy code that can use both forms, backfill, then remove old schema in a later release. Avoid a migration that instantly breaks the previous container version during a rolling deployment.

16.4 Versioning

- Semantic version tags for public releases.
- Database migrations are immutable after merge.
- API remains /api/v1 until an intentionally breaking contract requires v2.
- Container image tags include commit SHA; do not deploy mutable latest alone.

17. Backup, Disaster Recovery and Operations

17.1 Backup policy

- Enable automated RDS backups and point-in-time recovery.
- Take a manual snapshot before high-risk schema migrations.
- Redis is not the sole durable store; schedules and business state are reconstructible from PostgreSQL.
- S3 artifacts, if enabled, use lifecycle rules and encryption.
- Secrets configuration is documented but secret values are not placed in repository backups.

17.2 Recovery objectives

Objective	Production target
RPO	<= 15 minutes for database data when PITR is configured
RTO	<= 2 hours for a documented regional/service recovery exercise
Redis loss	Recover queue service and reconstruct future scheduled jobs from PostgreSQL
Worker/API container loss	ECS replaces unhealthy tasks automatically

17.3 Operational runbooks

- API 5xx spike: inspect CloudWatch trace/logs, DB connections and recent deployment; rollback if correlated.
- Execution queue growing: inspect oldest job age, worker errors, target throttling and Redis; scale workers within concurrency limits.
- Scheduler lag: verify reconciler lock, DB next_run_at query and Redis health; run reconciliation.
- Alert delivery failures: inspect provider status/credentials, retry queue and channel configuration.
- Database high connections: inspect pool sizes, long queries and worker scaling; reduce per-task pools before adding capacity.
- Suspected credential leakage: revoke affected API key/secret, rotate encryption/infrastructure credentials and review audit logs.

18. Environment Strategy and Cost Control

Environment	Purpose	Recommended footprint
Local	Daily development	Docker Compose Postgres/Redis + local services
CI	Automated tests	Ephemeral service containers
Staging	Production-like validation	Small AWS or managed equivalents; separate DB/Redis/secrets
Production	Public portfolio/real deployment	AWS architecture in this report, sized conservatively and scaled from metrics

18.1 Cost-conscious portfolio profile

For a personal portfolio, the architecture can preserve service separation while using smaller managed products: static frontend on S3/CloudFront, one small container host or low-count ECS tasks for API/worker, single-AZ small PostgreSQL, small Redis or a compatible managed Redis service, and minimal log retention. The production profile should remain documented even if the live demo uses the cheaper footprint.

Cost warning

In a fully private AWS VPC, NAT gateways and always-on managed databases can dominate a student/demo bill. Treat cost as an architecture constraint: use budgets/alarms, small instances/tasks, log retention, lifecycle policies and a staging environment that can be shut down when not in use. Verify current AWS pricing at deployment time.

19. Implementation Roadmap

Phase	Deliverables	Exit condition
Phase 1 - Foundation	Monorepo, auth, organizations/RBAC, PostgreSQL/Prisma, basic React shell	User can create org/project and access is tenant-isolated
Phase 2 - Test engine	Test builder, variables, manual runs, execution worker, core assertions	Manual test reliably produces stored execution + assertion detail
Phase 3 - Scheduling	Redis/BullMQ, delayed jobs, reconciler, rate limits, history	Schedules survive service restart/Redis rebuild scenario

Phase	Deliverables	Exit condition
Phase 4 - Reliability UX	Dashboards, uptime/latency aggregates, incidents, Slack/email/webhook alerts, live events	A known failing endpoint opens incident and sends deduplicated alert, recovery resolves it
Phase 5 - Developer tooling	OpenAPI import, public API key, CLI/GitHub Actions integration, audit log	CI can trigger collection and fail pipeline on assertion failure
Phase 6 - Production hardening	SSRF controls, redaction, load/security tests, Docker, AWS CDK, CI/CD, backups/alarms	Staging passes release gate and production smoke test

19.1 Build priority

Do not begin with dashboards. Build the execution engine and durable job model first. A beautiful dashboard backed by unreliable scheduling is still an unreliable system wearing nice CSS.

20. Definition of Done and Acceptance Criteria

20.1 Product acceptance

- [] User can register/login and create an organization; cross-tenant access tests pass.
- [] User can create REST/GraphQL/SOAP-style HTTP tests with variables, auth and required assertions.
- [] Manual execution returns quickly with a queued run and completes asynchronously.
- [] Scheduled tests continue across API/worker restarts and can be reconstructed after Redis data loss.
- [] Execution detail shows request metadata, redacted response preview, latency and individual assertion results.
- [] Dashboard shows current state, uptime and p50/p95/p99 latency for selectable ranges.
- [] Incident opens according to failure threshold, deduplicates, alerts, and resolves after configured successes.
- [] Slack/email/webhook test and real alert delivery have visible status/retry history.
- [] OpenAPI import creates draft tests without silently enabling production monitoring.
- [] Scoped API key/CLI can trigger a collection run from CI and return pass/fail.
- [] SSRF test suite blocks local/private/link-local/metadata and redirect-to-private targets.
- [] Authorization/cookie/secret values do not appear in application logs or stored previews.
- [] Dockerized staging deployment comes from source control and automated pipeline.
- [] CloudWatch alarms, RDS backups and rollback process are verified.
- [] Repository README contains architecture, local setup, environment variables, demo credentials/process and screenshots.

20.2 Portfolio acceptance

- Public demo domain available over HTTPS.
- A demo project contains at least one healthy, one slow/degraded and one intentionally failing API.
- README includes architecture diagram and a 2-3 minute demo flow.
- GitHub Actions pipeline visible in repository.
- No production secrets committed to Git history.

21. Risk Register

Risk	Impact	Mitigation
SSRF through user-configured URLs	Critical	Mandatory IP/DNS/redirect validation, worker isolation, egress policy and regression tests
Queue duplicate/lost work	High	DB source of truth, deterministic job IDs, idempotency uniqueness and reconciliation
Secrets leaked in logs/responses	Critical	Redaction middleware, encrypted secrets, preview minimization, security tests
External APIs rate-limit Monitor-X	Medium	Per-host/tenant limits, configurable frequency, backoff and transparent result classification
Database grows rapidly	Medium	Retention, pagination, indexes, aggregation and optional partitioning
Alert storms	Medium	Incident dedupe, cooldown and state-change based notifications
High AWS demo cost	Medium	Small footprint, budgets, log retention, lifecycle, optional lower-cost profile
Over-expanding scope	High	Version 1 boundaries and phased roadmap; browser testing/APM/global probes remain out of scope
Team member changes insecurely	Medium	RBAC, audit logs, owner controls and immediate key/session revocation

22. CV / Portfolio Positioning

When the implementation matches this specification, Monitor-X is no longer a basic college CRUD application. It demonstrates backend design, distributed queues, safe outbound networking, multitenancy, reliability engineering, cloud deployment and developer tooling.

22.1 Suggested CV bullets after completion

- Engineered Monitor-X, a production-style API testing and reliability platform supporting reusable response assertions, scheduled health checks, historical uptime/latency analytics, incident tracking and real-time alerts.
- Designed an asynchronous execution architecture using Redis and BullMQ with distributed workers, deterministic job IDs, retries/backoff, rate limiting and PostgreSQL-backed schedule reconciliation.
- Implemented multi-tenant RBAC, encrypted environment secrets, SSRF-safe outbound request controls, OpenAPI imports, Slack/email/webhook alerts and CI/CD-triggered collection runs.
- Containerized and deployed the platform on AWS using ECS, RDS PostgreSQL, ElastiCache, S3/CloudFront, ECR, Secrets Manager/KMS and CloudWatch with automated CI/CD and rollback.

CV rule

Only claim technologies and production behaviors that are actually implemented and demonstrable in the repository/deployment. The report defines the target; the final CV should describe the finished system, not the plan.

Appendix A - Environment Variable Catalog

Variable	Service	Sensitivity / purpose
NODE_ENV	api/worker	Environment name
APP_BASE_URL	api	Web origin for links/CORS
API_BASE_URL	web	Public API origin
DATABASE_URL	api/worker	Secret PostgreSQL connection string
REDIS_URL	api/worker	Secret Redis endpoint/credentials
JWT/SESSION_SECRET	api	Secret session signing/encryption material
KMS_KEY_ID	api/worker	KMS key reference

Variable	Service	Sensitivity / purpose
TENANT_ENCRYPTION_CONTEXT	api/worker	Encryption namespace/context
GITHUB_CLIENT_ID	api	OAuth identifier
GITHUB_CLIENT_SECRET	api	Secret OAuth credential
EMAIL_FROM	alert worker	Sender identity
SES_REGION / provider key	alert worker	Email provider configuration
OTEL_EXPORTER_*	api/worker	OpenTelemetry exporter
LOG_LEVEL	all	Structured logging level
MAX_RESPONSE_BYTES	worker	Outbound response safety limit
DEFAULT_TEST_TIMEOUT_MS	worker	Default execution timeout
WORKER_CONCURRENCY	worker	Per-task job concurrency
SSRF_ALLOW_PRIVATE_NETWORKS	worker	MUST remain false in public SaaS profile

Appendix B - Local Docker Compose Skeleton

```

services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: monitorx
      POSTGRES_USER: monitorx
      POSTGRES_PASSWORD: monitorx_dev
    ports: ["5432:5432"]
    volumes: ["pgdata:/var/lib/postgresql/data"]

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]

  mailpit:
    image: axllent/mailpit
    ports: ["8025:8025", "1025:1025"]

volumes:
  pgdata: {}

```

Pin tested image versions in the real repository and keep development passwords out of production configurations.

Appendix C - GitHub Actions Deployment Skeleton

```

name: deploy
on:
  push:
    branches: [main]

jobs:
  verify:
    steps:
      - checkout
      - setup Node + pnpm
      - pnpm install --frozen-lockfile
      - pnpm lint
      - pnpm typecheck
      - pnpm test
      - pnpm build

  deploy:

```

```
needs: verify
steps:
  - authenticate to AWS using OIDC
  - build API and worker images
  - push immutable commit-SHA tags to ECR
  - run database migration task
  - update ECS services
  - upload web build to S3
  - invalidate CloudFront
  - run smoke tests
```

Appendix D - Example Test Definition

```
{
  "name": "Create payment",
  "method": "POST",
  "urlTemplate": "{{BASE_URL}}/payments",
  "headers": {
    "Authorization": "Bearer {{PAYMENTS_TOKEN}}",
    "Content-Type": "application/json"
  },
  "body": {
    "amount": 500,
    "currency": "INR"
  },
  "timeoutMs": 8000,
  "assertions": [
    {"type": "status", "operator": "eq", "value": 201},
    {"type": "json_exists", "path": "$.paymentId"},
    {"type": "json_eq", "path": "$.status", "value": "created"},
    {"type": "latency", "operator": "lt", "valueMs": 700}
  ]
}
```

Appendix E - Production Deployment Checklist

- ☐ Domain and TLS certificate active
- ☐ Production secrets in Secrets Manager; no .env uploaded
- ☐ RDS private, backups/PITR enabled
- ☐ Redis private and reachable only from app security groups
- ☐ ECR images tagged with commit SHA
- ☐ ECS API readiness/liveness checks passing
- ☐ Execution workers can reach approved public API targets but not private/link-local ranges
- ☐ WAF/ALB/CloudFront configured
- ☐ Database migrations applied and recorded
- ☐ Frontend uses HTTPS API origin
- ☐ Slack/email/webhook test notifications succeed
- ☐ CloudWatch alarms created
- ☐ Budget/cost alarm created
- ☐ Controlled failing monitor opens incident and recovery resolves it
- ☐ Backup restore/recovery procedure documented
- ☐ Rollback to previous image tested
- ☐ README/demo project/screenshots published

Appendix F - Traceability to the Original Synopsis

The original synopsis is retained as the requirements baseline. The following mapping shows how its major areas are realized in the final architecture.

Original synopsis capability	Final implementation section
API test configuration	Sections 3.3-3.6, 10
Automated execution	Sections 6-7
Scheduled API testing	Sections 3.8 and 7
API health / response-time monitoring	Sections 3.9 and 12
Failure detection and logging	Sections 3.10 and 12
Dashboard and historical visualization	Sections 10 and 12
Alerting	Sections 3.11 and 12
Database storage	Section 8
Security requirements	Sections 6.3 and 11
Performance/scalability	Sections 4, 7 and 15
Deployment architecture	Sections 15-17

End of specification.